



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Name: ETHO MBA Steeve Rayanne

ID: 25979

ASSIGNMENT :

1/ The functional requirements :

ID	Functional Requirement
FR1	Employee Management: The system shall allow adding, updating, viewing, and deleting employee details (e.g., ID, name, role, base salary).
FR2	Payment Processing: The system shall calculate and process salary payments, taking into account base pay, bonuses, and tax/deductions.
FR3	Reporting: The system shall generate summary reports detailing total salary expenses and employee lists.

2/ Java Implementation:

I used Eclipse.

```
package com.auca;  
  
import java.util.ArrayList;  
import java.util.List;
```

```

public class EmployeeManagementSystem {

    public static class Employee {
        private String id;
        private String name;
        private double baseSalary;

        public Employee(String id, String name, double baseSalary) {
            this.id = id;
            this.name = name;
            this.baseSalary = baseSalary;
        }

        public String getId() { return id; }
        public String getName() { return name; }
        public double getBaseSalary() { return baseSalary; }
    }

    private List<Employee> employees = new ArrayList<>();

    // FR-01: Employee Management
    public boolean addEmployee(String id, String name, double baseSalary) {
        if (id == null || id.isEmpty() || baseSalary < 0) {
            return false;
        }
        employees.add(new Employee(id, name, baseSalary));
        return true;
    }

    // FR-02: Payment Processing
    public double processPayment(String employeeId, double bonus) {
        if (bonus < 0) {
            return -1.0;
        }

        Employee emp = null;
        for (Employee e : employees) {
            if (e.getId().equals(employeeId)) {
                emp = e;
                break;
            }
        }

        if (emp == null) {
            return -1.0;
        }

        double grossSalary = emp.getBaseSalary() + bonus;
        double taxRate;

        // Fault injected intentionally (>= instead of >)
        if (grossSalary >= 5000) {
            taxRate = 0.20;
        } else {

```

```

        taxRate = 0.10;
    }

    return grossSalary * (1 - taxRate);
}

// FR-03: Reporting
public String generateSummaryReport() {
    double totalExpense = 0;
    for (Employee e : employees) {
        totalExpense += e.getBaseSalary();
    }
    return "Total Employees: " + employees.size() + ", Total Base Expense: $" +
totalExpense;
}

// ENTRY POINT NEEDED BY ECLIPSE
public static void main(String[] args) {
    EmployeeManagementSystem ems = new EmployeeManagementSystem();

    // Add test employee
    ems.addEmployee("EMP001", "Steeve Rayanne", 4000);

    // Execute test case
    double actualNetPay = ems.processPayment("EMP001", 1000);
    double expectedNetPay = 4500.0;

    System.out.println("=== TEST EXECUTION RESULTS ===");
    System.out.println("Expected Net Pay: $" + expectedNetPay);
    System.out.println("Actual Net Pay:   $" + actualNetPay);

    if (actualNetPay == expectedNetPay) {
        System.out.println("TEST RESULT: PASS");
    } else {
        System.out.println("TEST RESULT: FAIL");
    }

    System.out.println("\n=== SUMMARY REPORT ===");
    System.out.println(ems.generateSummaryReport());
}
}

```

3/ General Test Information

Parameter	Information
Institution	Adventist University of Central Africa (AUCA)
Student Name	ETHO MBA Steeve Rayanne
Student ID	25979
Test Date	September 22, 2026
System Date	September 22, 2026
Hardware	Standard PC / x86 Workstation
Software	Java SE Development Kit (JDK 17 or higher), Eclipse IDE, JUnit 5, JaCoCo Coverage Tool

4/ Black-Box Test Case

Black-box testing focuses on testing inputs and expected outputs without looking at the internal source code structure.

ID	DESCRIPTION	REQUIREMENT TESTED	INPUT/PROCEDURE	EXPECTED RESULT	ACTUAL RESULT	RESULT
B B-01	Add a valid employee	FR1	ID="EMP001", Name="Steeve Rayanne", BaseSalary=4000	Employee added successfully(returns true)	Employee added successfully(returns true)	PASS

B B-02	Add employee with empty ID	FR1	ID="", Name="Steeve Rayanne", BaseSalary=4000	Employee registration failed (returns false)	Employee registration failed (returns false)	PASS
B B-03	Add employee with null ID	FR1	ID=null, Name="Steeve Rayanne", BaseSalary=4000	Employee registration failed (returns false)	Employee registration failed (returns false)	PASS
B B-04	Add employee with negative base salary	FR1	ID="EMP002", Name="Daniel Lawson", BaseSalary=-1000	Employee registration failed (returns false)	Employee registration failed (returns false)	PASS
B B-05	Process payment with negative bonus	FR2	ID="EMP001", Bonus=-100	Payment processing failed (returns -1.0)	Payment processing failed (returns -1.0)	PASS
B B-06	Process payment for non-existent employee ID	FR2	ID="EMP999", Bonus=500	Payment processing failed (returns -1.0)	Payment processing failed (returns -1.0)	PASS

B B-07	Process payment for gross salary under \$5,000	FR2	ID="EMP001" (Salary=4000) , Bonus=500 \$\\rightarrow\$ Gross=\$4500	10% tax applied, Net Pay = \$4,050.00	10% tax applied, Net Pay = \$4,050.00	PASS
B B-08	Process payment for boundary gross salary of \$5,000	FR2	ID="EMP001" (Salary=4000) , Bonus=1000 \$\\rightarrow\$ Gross=\$5000	10% tax applied, Net Pay = \$4,500.00	20% tax applied, Net Pay = \$4,000.00	FAIL
B B-09	Generate summary report when employees exist	FR3	Register EMP001 (Salary=4000) , then generate report	"Total Employees: 1, Total Base Expense: \$4000.0"	"Total Employees: 1, Total Base Expense: \$4000.0"	PASS

4/ White Box Test Case

White-box testing examines the internal structure, branches, and code logic.

ID	Description	Requirement	Internal condition/branch tested	Input / procedure	Expected result	Actual Result	Result
W B-01	Test null ID branch in addEmployee	FR1	if (id == null) branch	Call addEmployee(null, "Test", 3000)	Returns false without adding employee	Returns false	PASS

ID	Description	Requirement	Internal condition/branch tested	Input / procedure	Expected result	Actual Result	Result
W B-02	Test empty ID branch in addEmployee	FR1	if (id.isEmpty()) branch	Call addEmployee("", "Test", 3000)	Returns false without adding employee	Returns false	PASS
W B-03	Test negative salary branch in addEmployee	FR1	if (baseSalary < 0) branch	Call addEmployee("EMP01", "Test", -500)	Returns false without adding employee	Returns false	PASS
W B-04	Test negative bonus branch in processPayment	FR2	if (bonus < 0) branch	Call processPayment("EMP01", -50)	Returns -1.0	Returns -1.0	PASS
W B-05	Test employee lookup loop match branch	FR2	if (e.getId().equals(employeeId)) match found	Add "EMP001", then call processPayment("EMP01", 100)	Loop breaks, emp set correctly, calculates payment	Payment calculated correctly	PASS
W B-06	Test employee lookup null branch	FR2	if (emp == null) branch	Call processPayment("NOT_EXIST", 100)	Returns -1.0	Returns -1.0	PASS
W B-07	Test tax decision boundary condition (grossSalary >= 5000)	FR2	Relational operator branch evaluation on boundary	Gross = 5000 (grossSalary > 5000 evaluates to false)	Evaluates false, applies 10% tax (taxRate = 0.10) DOCX	Evaluates true, applies 20% tax (taxRate = 0.20) DOCX	FAIL

ID	Description	Requirement	Internal condition/branch tested	Input / procedure	Expected result	Actual Result	Result
WB-08	Test gross salary over \$5,000 branch	FR2	if (grossSalary >= 5000) True branch	Gross = 6000	Evaluates true, applies 20% tax (taxRate = 0.20), Net = \$4,800.00	Evaluates true, applies 20% tax (taxRate = 0.20), Net = \$4,800.00	PASS
WB-09	Test summary report loop execution	FR3	for (Employee e : employees) loop iteration	Add 2 employees (\$4000 and \$3000), call generateSummaryReport()	Loop sums both salaries, returning \$7000 total expense	Returns total expense \$7000.0	PASS

GitHub Repository : https://github.com/rayanne25/Software_Testing_Assignment1.git